

Sprawozdanie z projektu

Steganografia w LLM

Kamil Pinas, Filip Połatyński

26 kwietnia 2026 r.

Streszczenie

Celem projektu jest ukrywanie wiadomości tekstowej w naturalnie brzmiącym tekście generowanym przez model językowy TinyLlama. Zastosowano deterministyczny mechanizm wyboru tokenów, w którym bity sekretu są odwzorowywane na pozycję tokenu w przetasowanej liście kandydatów. Dzięki temu możliwe jest późniejsze odtworzenie wiadomości przez dekoder działający na tych samych parametrach i z tym samym hasłem. Projekt zawiera również interfejs graficzny napisany w Qt for Python, który ułatwia wprowadzanie danych, uruchamianie enkodowania i dekodowania oraz podgląd logów.

1 Wstęp

Steganografia polega na ukrywaniu samego faktu istnienia tajnej wiadomości. W odróżnieniu od klasycznego szyfrowania, którego celem jest zabezpieczenie treści, steganografia dąży do tego, aby komunikat nie wzbudzał podejrzeń. W tym projekcie nośnikiem informacji jest tekst generowany przez model językowy, a ukrywanie danych odbywa się na poziomie doboru kolejnych tokenów.

Projekt stanowi praktyczną demonstrację, że model generatywny może zostać wykorzystany nie tylko do tworzenia naturalnego tekstu, lecz także do przenoszenia dodatkowego, ukrytego ładunku informacyjnego.

2 Opis działania systemu

System składa się z trzech głównych warstw: modułu enkodera, modułu dekodera oraz warstwy graficznej. Enkoder i dekoder korzystają z tego samego modelu językowego, tych samych parametrów próbkowania oraz tego samego hasła, które inicjalizuje deterministyczny generator pseudolosowy.

Dane wejściowe i wyjściowe są przechowywane w plikach JSON. Najważniejsze z nich to:

- `data/input.json` – dane wejściowe użytkownika,
- `data/config.json` – wspólna konfiguracja parametrów,
- `data/message.json` – wygenerowana wiadomość steganograficzna,
- `data/decoded_message.json` – odzyskany sekret.

2.1 Kodowanie wiadomości

Proces kodowania rozpoczyna się od zamiany sekretu na ciąg bitów. Do wiadomości dołączany jest także znacznik końca komunikatu **EOM** o wartości `\x04`. Dzięki temu dekodér wie, kiedy zakończyć odtwarzanie danych.

W każdym kroku generacji enkoder pobiera z modelu listę tokenów o najwyższym prawdopodobieństwie. Następnie:

1. odrzuca tokeny o zbyt małym prawdopodobieństwie na podstawie progu **threshold**,
2. wyznacza liczbę dostępnych tokenów,
3. oblicza pojemność kroku jako największą potęgę dwójki nieprzekraczającą liczby tokenów,
4. pobiera odpowiednią liczbę bitów z wiadomości,
5. tasuje listę tokenów za pomocą generatora losowego ziarna wyprowadzonego z hasła,
6. wybiera token znajdujący się na pozycji odpowiadającej wartości pobranych bitów.

Taki mechanizm gwarantuje, że ten sam sekret i to samo hasło prowadzą do powstania identycznej ścieżki wyboru tokenów. Zależność od top-N i progu prawdopodobieństwa sprawia jednak, że rzeczywista pojemność może się zmieniać w zależności od kontekstu generacji.

Warto zauważyć, że przy pojemności równej 1 (czyli gdy po filtrowaniu pozostaje tylko jedna sensowna opcja) w danym kroku nie da się zakodować żadnego bitu. Wtedy model wykonuje krok generacji, ale licznik zakodowanych bitów nie rośnie.

Po zakodowaniu całego ciągu bitów (sekret + **EOM**) enkoder przechodzi do fazy domykania wypowiedzi. W tej fazie:

- nie koduje już kolejnych bitów,
- wybiera najbardziej prawdopodobny token kontynuacji, aby tekst pozostawał naturalny,
- monitoruje prawdopodobieństwo tokenu końca sekwencji (**EOS**).

Generowanie kończy się, gdy prawdopodobieństwo **EOS** osiągnie próg **eos_threshold**. Oznacza to, że model jest już gotowy semantycznie zamknąć wypowiedź i dalsza kontynuacja nie jest wymagana do przeniesienia danych ukrytych.

2.2 Dekodowanie wiadomości

Dekoder odtwarza sekret, przetwarzając zapisany wcześniej plik **message.json**. Działa on w trybie symetrycznym względem enkodera: odtwarza identyczne kroki generacji, tworzy tę samą listę kandydatów i wykonuje to samo tasowanie zależne od hasła. Następnie odczytuje pozycję rzeczywiście wybranego tokenu w przetasowanej liście i przekształca ją z powrotem na bity.

Po zebraniu kolejnych fragmentów wiadomości dekodér składa je w znaki ASCII i zatrzymuje się po wykryciu znacznika końca wiadomości. Wynik zapisywany jest do **data/decoded_message.json**.

2.3 Parametry sterujące i ich znaczenie

Najważniejsze parametry wejściowe i runtime zestawiono w tabeli poniżej.

Parametr	Znaczenie
<code>prompt</code>	Tekst początkowy przekazywany do modelu. Definiuje kontekst generacji.
<code>secret</code>	Wiadomość ukrywana w wyborach tokenów. Jest konwertowana do bitów i rozszerzana o znacznik końca EOM.
<code>password</code>	Hasło używane do deterministycznego ziarna PRNG. Musi być identyczne przy enkodowaniu i dekodowaniu, inaczej odtworzenie bitów nie powiedzie się.
<code>threshold</code>	Minimalne prawdopodobieństwo tokenu dopuszczonego do kodowania. Wyższa wartość zwykle zmniejsza pojemność (mniej bitów na krok), ale poprawia naturalność tekstu.
<code>eos_threshold</code>	Próg zakończenia generacji po zakodowaniu całego sekretu. Gdy prawdopodobieństwo tokenu EOS osiągnie ten poziom, enkoder zatrzymuje generowanie.
<code>top_n</code>	Liczba najwyżej ocenianych tokenów analizowanych w każdym kroku.

Table 1: Parametry sterujące i ich wpływ na działanie systemu

3 Architektura aplikacji i GUI

Warstwa użytkowa została zrealizowana w Qt for Python. Plik `app.py` pełni rolę mostu pomiędzy interfejsem QML a logiką Pythona. Klasa `PythonBridge` udostępnia metody wywoływane z GUI, takie jak generowanie zakodowanego tekstu, odczyt sekretu i aktualizacja parametrów pracy.

Interfejs składa się z głównego pliku `AppUI.qml` oraz modułów umieszczonych w katalogu `components/`. Zastosowano podział na panele:

- panel ustawień runtime,
- panel kodowania,
- panel dekodowania,
- panel terminala z logami.

Takie rozdzielenie upraszcza korzystanie z aplikacji i pozwala użytkownikowi szybko zmieniać parametry bez edycji plików JSON. Dodatkowo standardowe wyjście i błędy są przekierowywane do terminala w aplikacji, co ułatwia obserwację przebiegu działania modelu.

3.1 Widok aplikacji

Na rysunku poniżej przedstawiono aktualny interfejs GUI aplikacji.

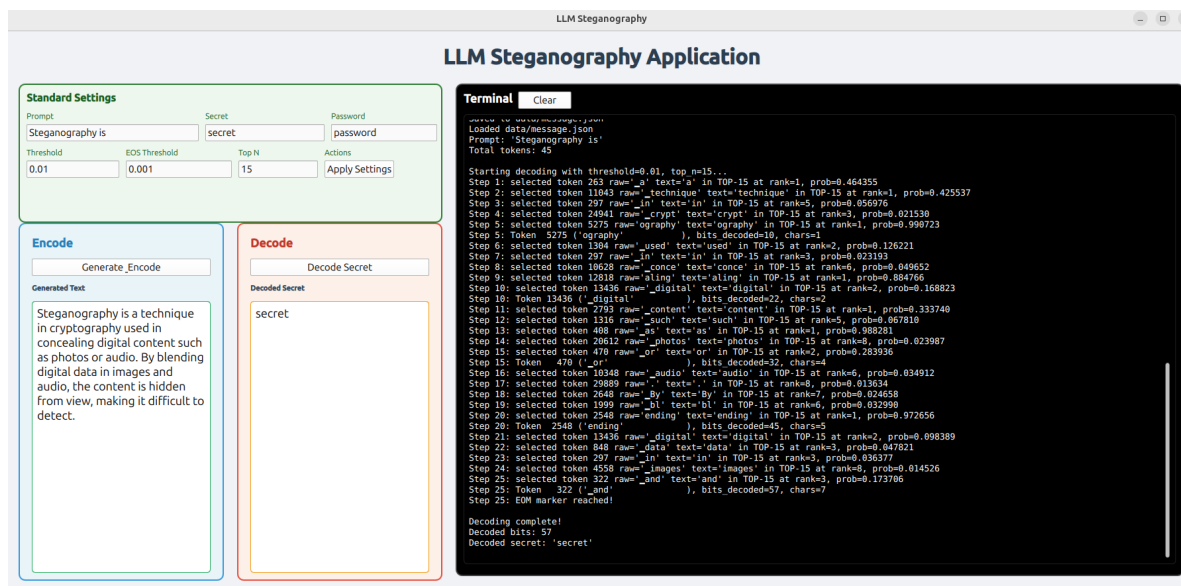


Figure 1: Widok aplikacji GUI z panelami Encode/Decode, ustawieniami oraz terminalem logów

3.2 Materiał demonstracyjny (wideo)

Prezentacja działania aplikacji jest dostępna pod adresem:

[Film demonstracyjny projektu \(Google Drive\)](#)

4 Uruchomienie

Projekt można uruchomić lokalnie po zainstalowaniu zależności z pliku `requirements.txt`. Typowy scenariusz użycia wygląda następująco:

1. uruchomienie aplikacji GUI poleceniem `python app.py`,
2. wpisanie promptu, sekretu, hasła i parametrów modelu,
3. wygenerowanie tekstu steganograficznego,
4. uruchomienie dekodowania i odczyt sekretu z pliku wyjściowego.

W trybie tekstowym możliwe jest również bezpośrednie użycie modułów `encoder.py` i `decoder.py`, co ułatwia testowanie samego algorytmu bez GUI.

5 Ograniczenia

Rozwiązanie ma kilka istotnych ograniczeń:

- nie zapewnia ochrony kryptograficznej – bezpieczeństwo opiera się wyłącznie na ukryciu treści,
- dekodowanie wymaga dokładnie tych samych parametrów, co kodowanie,
- pojemność zależy od aktualnego rozkładu prawdopodobieństw tokenów,
- zmiana modelu lub parametrów próbkowania może uniemożliwić poprawne odtworzenie wiadomości,

6 Wnioski

Zrealizowany system pokazuje, że generatywny model językowy może posłużyć jako nośnik ukrytej wiadomości, jeśli wybór tokenów zostanie odpowiednio zdeterministycznie zakodowany. Najważniejszą cechą rozwiązania jest spójność między enkoderem i dekodorem: oba komponenty muszą wykonywać identyczne kroki, aby możliwe było odzyskanie sekretu.

Z perspektywy projektu osiągnięto zarówno część algorytmiczną, jak i użytkową. Algorytm demonstruje ideę steganografii w tekście generowanym przez LLM, a GUI upraszcza korzystanie z aplikacji i pozwala wygodnie sprawdzać działanie systemu.

References

- [1] Dokumentacja projektu: *README.md*.
- [2] Dokumentacja GUI: *PYQT_README.md*.
- [3] Model wykorzystywany w projekcie: TinyLlama-1.1B-Chat-v1.0.
- [4] Film demonstracyjny projektu: <https://drive.google.com/file/d/19c5W6C8ZqfG-WvP81zaWk9ezISu9Fe-R/view?usp=sharing>.